

Cận dưới và các mô hình tính toán khác (Lower Bounds and Other Models)

PHỤ THUỘC A/03 cho mô hình chi phí và phân cấp bộ nhớ; A/05 cho lập luận đếm; D/23 cho mặt kia của độ khó.

TÓM TẮT

Chương trước nói về những bài toán mà *chưa ai* tìm được lời giải nhanh. Chương này nói về hai thứ khác: những trường hợp ta *chứng minh được* rằng không thể nhanh hơn, và những mô hình tính toán mà trong đó bảng xếp hạng thuật toán thay đổi hoàn toàn. Nửa đầu trình bày ba kỹ thuật chứng minh cận dưới — cây quyết định, lập luận đối thủ, và đếm thông tin — cùng lý do vì sao cận dưới lại khó chứng minh tới vậy. Nửa sau bỏ giả định lớn nhất của cả quyển sách: rằng dữ liệu nằm vừa trong bộ nhớ, đọc được nhiều lần, và máy chỉ có một lõi. Khi giả định đó sai — bộ nhớ ngoài, luồng dữ liệu, trực tuyến, song song — thì thuật toán tốt nhất đổi hẳn. Nếu chỉ nhớ một điều: mô hình chi phí quyết định thuật toán, nên khi kết quả thực nghiệm mâu thuẫn với lý thuyết, thủ phạm gần như luôn là mô hình chứ không phải phép đo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **cận dưới** — phát biểu rằng *mọi* thuật toán trong một mô hình nào đó đều cần ít nhất một lượng tài nguyên nhất định. Khác hẳn với việc phân tích chi phí của một thuật toán cụ thể.
- **cây quyết định** — mô hình trong đó thuật toán chỉ được đặt câu hỏi so sánh; mỗi lá là một kết quả khả dĩ, nên số lá bị chặn dưới bởi số kết quả.
- **lập luận đối thủ** — kỹ thuật chứng minh cận dưới bằng cách hình dung một đối thủ trả lời các câu hỏi của thuật toán một cách bất lợi nhất, miễn là vẫn nhất quán.
- **mô hình bộ nhớ ngoài** — tính chi phí theo số lần chuyển khối giữa bộ nhớ nhanh và chậm, thay vì theo số phép toán.
- **thuật toán luồng dữ liệu** — đọc dữ liệu một hoặc vài lượt, dùng bộ nhớ nhỏ hơn dữ liệu rất nhiều; thường phải chấp nhận kết quả xấp xỉ.
- **tỉ lệ cạnh tranh** — với thuật toán trực tuyến, tỉ số giữa chi phí của nó và chi phí của lời giải tối ưu biết trước toàn bộ tương lai.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chứng minh “không thuật toán nào nhanh hơn” lại khó hơn nhiều so với việc tìm một thuật toán nhanh?
- Q2. Cận dưới $\Omega(n \log n)$ cho sắp xếp có mâu thuẫn với việc sắp xếp đếm chạy $O(n)$ không?
- Q3. Trong mô hình bộ nhớ ngoài, vì sao một thuật toán “chậm hơn” về số phép toán lại có thể nhanh hơn nhiều lần?
- Q4. Thuật toán trực tuyến không biết tương lai. Vậy ta đánh giá nó bằng thước đo nào cho công bằng?
- Q5. Vì sao một chương trình song song trên 32 lõi thường không nhanh gấp 32 lần?

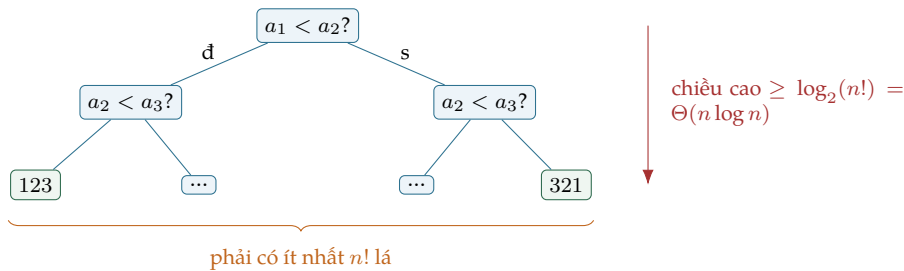
Toàn bộ quyển sách này, tới chương trước, đi theo một hướng: cho một bài toán, tìm thuật toán nhanh hơn. Chương này đi ngược lại và hỏi: *có điểm dừng không, và làm sao biết mình đã tới đó?*

Câu hỏi ấy có giá trị thực dụng lớn hơn vẻ ngoài học thuật của nó. Nếu bạn biết một bài toán cần ít nhất $\Omega(n \log n)$, bạn ngừng tìm lời giải tuyến tính. Nếu bạn biết mọi thuật toán so sánh đều cần $\Omega(n \log n)$ nhưng bài của bạn có thêm thông tin — khoá là số nguyên nhỏ — thì bạn biết chính xác phải phá vỡ giả định nào để đi nhanh hơn. Cận dưới không chỉ nói “dừng lại” mà còn nói *dừng ở đâu và vì sao*.

Nửa sau chương làm một việc khác nhưng cùng tinh thần: nó chỉ ra rằng nhiều kết luận trong các chương trước là hệ quả của một mô hình chi phí cụ thể, và khi mô hình đổi thì kết luận đổi theo.

1.1 Ba cách chứng minh cận dưới

Cách thứ nhất — cây quyết định. Nếu thuật toán chỉ được phép so sánh các phần tử, thì quá trình chạy của nó là một đường đi trên một cây nhị phân: mỗi nút là một phép so sánh, mỗi lá là một kết luận. Với bài sắp xếp n phần tử, số kết luận khả dĩ là $n!$ hoán vị, nên cây phải có ít nhất $n!$ lá, nên chiều cao ít nhất $\log_2(n!) = \Theta(n \log n)$. Đó là cận dưới nổi tiếng nhất trong ngành, và nó chỉ tồn tại ba câu để chứng minh — nhờ công thức Stirling ở chương A/05.



Hình 1.1. Cận dưới cho sắp xếp bằng cây quyết định. Mỗi phép so sánh chia đôi tập kết quả còn khả dĩ, mà số kết quả là $n!$, nên cây phải cao ít nhất $\log_2(n!)$. Điều bắt buộc phải nhớ kèm: kết luận này chỉ đúng *trong mô hình so sánh* — sắp xếp đếm dùng giá trị làm chỉ số nên nằm ngoài mô hình và không mâu thuẫn.

Điểm mấu chốt, và cũng là câu trả lời cho câu hỏi mở đầu thứ hai: cận này chỉ đúng *trong mô hình so sánh*. Sắp xếp đếm không so sánh phần tử với nhau mà dùng giá trị của chúng làm chỉ số mảng — một thao tác mà mô hình cây quyết định không cho phép. Không có mâu thuẫn nào; chỉ có hai mô hình khác nhau. Đây là bài học tổng quát: *mọi cận dưới đều kèm một mô hình, và bỏ qua mô hình là hiểu sai kết quả*.

Cách thứ hai — lập luận đối thủ. Hình dung một đối thủ không quyết định dữ liệu từ trước mà trả lời từng câu hỏi của thuật toán, luôn chọn câu trả lời bất lợi nhất, miễn là vẫn nhất quán với mọi câu đã trả lời. Nếu đối thủ giữ được ít nhất hai khả năng cho tới sau k câu hỏi thì thuật toán không thể kết luận sớm hơn $k + 1$. Ví dụ: tìm đồng thời cực đại và cực tiểu của n phần tử cần ít nhất $\lceil 3n/2 \rceil - 2$ phép so sánh, chứng minh bằng cách đếm lượng thông tin mà mỗi phép so sánh có thể cung cấp.

Cách thứ ba — đếm hoặc lập luận thông tin. Nếu kết quả cần phân biệt N khả năng và mỗi bước cho tối đa một bit thông tin thì cần ít nhất $\log_2 N$ bước. Đây là dạng tổng quát của cách thứ nhất, và nó cũng là ngôn ngữ chung với lý thuyết thông tin — lý do mà cận dưới cho sắp xếp và cận dưới cho nén dữ liệu có cùng hình thức.

1.2 Vì sao cận dưới khó

Câu hỏi mở đầu thứ nhất đáng được trả lời kỹ, vì nó giải thích tình trạng của cả ngành.

Để chứng minh một thuật toán *tồn tại*, bạn chỉ cần đưa ra một thuật toán — một đối tượng cụ thể. Để chứng minh *không thuật toán nào* nhanh hơn, bạn phải lập luận về *tất cả* các thuật toán khả dĩ, kể cả những cái chưa ai nghĩ ra. Đó là một mệnh đề toàn thể trên một tập vô hạn các đối tượng rất tự do, và ta có rất ít công cụ để nói điều gì đó về mọi thành viên của tập đó.

Vì vậy các cận dưới đã chứng minh được hầu như đều nằm trong những mô hình *hạn chế*: chỉ được so sánh, chỉ được truy cập theo một kiểu nhất định, chỉ có một lượt đọc. Trong mô hình tính toán tổng quát, chúng ta gần như không chứng minh được cận dưới nào đáng kể — và đó chính là lý do câu hỏi P đối với NP vẫn mở sau nửa thế kỷ. Câu chuyện Karatsuba ở chương C/14 là lời cảnh báo cụ thể: một nhà toán học hàng đầu phỏng đoán rằng nhân số cần $\Omega(n^2)$, và phỏng đoán đó bị bác bỏ trong một tuần. Trục giác về cận dưới rất dễ sai.

Một hướng hiện đại đáng biết là *độ phức tạp tính*: thay vì cố chứng minh cận dưới tuyệt đối, người ta chứng minh cận dưới *có điều kiện*, dạng “nếu bài toán A không thể giải nhanh hơn thì bài toán B cũng không”. Nhờ đó có những kết quả kiểu “khoảng cách chỉnh sửa không thể tính nhanh hơn đáng kể so với $O(n^2)$, trừ khi một giả thuyết được tin rộng rãi bị bác bỏ”. Đây là cách ngành này sống chung với sự khó của cận dưới: xây một mạng lưới các kết quả liên hệ với nhau thay vì chờ một chứng minh tuyệt đối.

1.3 Độ phức tạp tính: cận dưới có điều kiện, và cách dùng chúng

Mục trên kết thúc bằng một câu về “độ phức tạp tính”. Ý tưởng ấy đáng được nói kỹ hơn, vì nó là hướng sống động nhất của lý thuyết cận dưới hiện nay và vì nó có ích ngay cả cho người chỉ đi thi.

Cách đặt vấn đề là như sau. Ta không chứng minh nổi cận dưới tuyệt đối, vậy hãy chọn một *giả thuyết neo* mà cộng đồng tin, rồi quy dẫn để cho thấy “nếu bài toán của tôi nhanh hơn thì giả thuyết neo sụp”. Giả thuyết neo hay dùng nhất là **giả thuyết thời gian mũ mạnh** (*strong exponential time hypothesis*, SETH): không có $\varepsilon > 0$ nào để mọi bài k -SAT giải được trong $(2 - \varepsilon)^n$ — một phát biểu ra đời từ Impagliazzo và Paturi[1]. Giả thuyết neo thứ hai là bài 3SUM: cho n số, có ba số nào cộng lại bằng 0 không — tin rằng không có thuật toán $n^{2-\varepsilon}$, dù các cải tiến ở mức thừa số lôgarit thì đã có.

Kết quả đáng nhớ nhất của hướng này, ít nhất với người đọc quyển sách này, là của Backurs và Indyk[2]: nếu tính được khoảng cách chỉnh sửa của hai chuỗi độ dài n trong $O(n^{2-\varepsilon})$ thì SETH sai. Nói cách khác, bảng quy hoạch động $O(nm)$ mà bạn viết ở chương C/16 — một bài tập nhập môn — gần như chắc chắn là đã tối ưu. Ba chục năm người ta cố cải tiến nó và không ai thành công; bây giờ ta biết *vì sao*.

Nhánh 3SUM cho một câu chuyện song song trong hình học. Gajentaan và Overmars[3] chỉ ra cả một họ bài toán phẳng — có ba điểm nào thẳng hàng không, hợp của các tam giác có lỗ không, một số bài lập kế hoạch chuyển động — đều khó ít nhất bằng 3SUM. Điều đó giải thích một hiện tượng mà người thi đấu hay gặp: rất nhiều bài hình học “chỉ” có lời giải n^2 và không ai tìm được gì tốt hơn.

Giá trị thực hành nằm ở chỗ ít ai nói ra: cận dưới có điều kiện cho bạn *quyền dùng tìm*. Khi nhận ra bài mình đang vật lộn quy dẫn được về khoảng cách chỉnh sửa hay về 3SUM, câu hỏi đúng không còn là “làm sao nhanh hơn về mặt tiệm cận” mà chuyển thành ba câu khác: hằng số có giảm được không, đề bài có ràng buộc riêng nào phá được cấu trúc khó không (chuỗi ngắn, bảng chữ cái nhỏ, các điểm nằm trên lưới), và có chấp nhận được lời giải xấp xỉ không. Đó cũng chính

là bốn cách sống chung ở chương D/23, chỉ khác là ở đây ranh giới không phải giữa đa thức và mũ, mà giữa n^2 và $n^{2-\varepsilon}$.

1.4 Mô hình bộ nhớ ngoài: khi đơn vị chi phí là lần đọc khối

Chương A/03 đã cảnh báo rằng mô hình RAM coi mọi truy cập bộ nhớ là như nhau, còn thực tế thì có phân cấp. Mục này lấy nhận xét đó làm mô hình chính thức.

Trong mô hình bộ nhớ ngoài, ta đếm số lần chuyển *khối* dữ liệu giữa bộ nhớ chậm và nhanh, mỗi khối chứa B phần tử, và bộ nhớ nhanh chứa được M phần tử. Với mô hình này, sắp xếp không còn là $O(n \log n)$ mà là $O(\frac{n}{B} \log_{M/B} \frac{n}{B})$ lần chuyển khối — và cơ số của lôgarit là M/B , thường hàng trăm, nên số “tầng” rất ít. Đó là nền lý thuyết cho sắp xếp trộn nhiều đường, và cũng là lý do B-cây có nút to ($B/09$).

Điều đáng chú ý nhất, và là câu trả lời cho câu hỏi mở đầu thứ ba: trong mô hình này, một thuật toán làm nhiều phép toán hơn nhưng truy cập *tuần tự* có thể thắng một thuật toán ít phép toán hơn nhưng nhảy ngẫu nhiên — vì chi phí thật nằm ở việc kéo khối dữ liệu, không ở việc tính toán.

Cache-oblivious là một ý tưởng đẹp trong nhánh này[4]: thiết kế thuật toán tối ưu về số lần chuyển khối mà *không cần biết* B và M là bao nhiêu. Kỹ thuật chung là chia để trị đệ quy tới mọi tỉ lệ, để ở tầng nào của phân cấp bộ nhớ thì cũng có một mức đệ quy vừa khít. Nhờ vậy một chương trình chạy tốt trên mọi máy mà không phải tinh chỉnh tham số.

1.5 Ba mô hình bỏ giả định khác

Luồng dữ liệu. Dữ liệu chảy qua một lần và quá lớn để lưu; bạn có bộ nhớ nhỏ hơn nhiều bậc. Trong mô hình này, hầu hết bài toán chính xác trở nên bất khả thi, và người ta chuyển sang xấp xỉ: đếm số phần tử phân biệt bằng HyperLogLog, tìm phần tử xuất hiện nhiều bằng sơ đồ đếm, lấy mẫu đều bằng hồ chứa ($C/20$). Đây là mô hình của các hệ thống đo lường và phân tích nhật ký ở quy mô lớn.

Trực tuyến. Bạn phải quyết định ngay khi mỗi dữ liệu tới, không được xét lại, và không biết tương lai. Câu hỏi mở đầu thứ tư hỏi về thước đo, và câu trả lời là *tỉ lệ cạnh tranh*: so chi phí của bạn với chi phí của một lời giải tối ưu *biết trước toàn bộ tương lai*. Ví dụ kinh điển là bài thuê hay mua: mỗi ngày trượt tuyết bạn có thể thuê ván với giá 1 hoặc mua hẳn với giá B , và bạn không biết mình sẽ trượt bao nhiêu ngày; chiến lược “thuê B ngày rồi mua” có tỉ lệ cạnh tranh 2, và người ta chứng minh được không chiến lược tắt định nào tốt hơn. Cùng khung phân tích áp cho các chính sách thay thế bộ nhớ đệm: LRU có tỉ lệ cạnh tranh phụ thuộc kích thước bộ đệm, và phân tích đó giải thích vì sao nó là lựa chọn hợp lý dù không tối ưu.

Song song. Bỏ giả định một lõi. Thước đo chuẩn gồm hai đại lượng: *tổng công việc* — số phép toán nếu chạy tuần tự — và *chiều sâu* — độ dài chuỗi phụ thuộc dài nhất, tức thời gian nếu có vô hạn bộ xử lý. Thời gian thực tế bị chặn dưới bởi chiều sâu, nên một thuật toán có chuỗi phụ thuộc dài thì song song hoá bao nhiêu cũng vô ích.

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó có ba lớp lý do. Thứ nhất, phần tuần tự không loại bỏ được sẽ chi phối khi số lõi tăng — nếu 5% chương trình là tuần tự thì dù có vô hạn lõi cũng không nhanh hơn 20 lần. Thứ hai, chi phí điều phối: tạo luồng, đồng bộ, truyền dữ liệu đều tốn, và chúng tăng theo số lõi. Thứ ba, tranh chấp tài nguyên chung, đặc biệt là băng thông bộ nhớ — nhiều lõi cùng đọc thì bộ nhớ trở thành nút cổ chai. Kết quả thực nghiệm nổi tiếng của

McSherry và cộng sự[5] cho thấy nhiều hệ thống song song bị một chương trình đơn luồng viết cẩn thận đánh bại, đúng vì ba lý do trên.

CẦN NHỚ

Mô hình chi phí quyết định thuật toán. Bốn mô hình trong chương này — bộ nhớ ngoài, luồng, trực tuyến, song song — mỗi cái làm một họ thuật toán khác nhau trở thành tốt nhất. Khi bạn thấy một thuật toán “tốt hơn về lý thuyết” lại thua trong thực nghiệm, câu hỏi đầu tiên không phải “phép đo có sai không” mà là “tôi đang ở mô hình chi phí nào”.

1.6 Gốc rễ: từ cân đo tới các hệ thống dữ liệu lớn

Cận dưới cho sắp xếp thuộc loại kết quả xuất hiện sớm và tự nhiên: các bài toán “cân bao nhiêu lần thì tìm ra đồng xu giả” đã có trong toán giải trí từ lâu, và chúng chính là lập luận cây quyết định ở dạng dân dã. Việc hình thức hoá thành một cận dưới cho sắp xếp thuộc về giai đoạn đầu của ngành, và nó vẫn là ví dụ chuẩn để dạy khái niệm “cận dưới trong một mô hình”.

Phân tích cạnh tranh cho thuật toán trực tuyến ra đời năm 1985 trong cùng bài báo mà Sleator và Tarjan giới thiệu splay tree[6] — một sự trùng hợp đáng chú ý, vì cả hai ý tưởng đều xuất phát từ cùng một câu hỏi: làm sao đánh giá công bằng một thuật toán không có đảm bảo cho từng thao tác riêng lẻ. Phân tích khấu hao trả lời cho trường hợp offline; phân tích cạnh tranh trả lời cho trường hợp không biết tương lai. Bối cảnh thúc đẩy là rất thực tế: các chính sách quản lý bộ nhớ đệm trong hệ điều hành cần một tiêu chuẩn đánh giá, và “trường hợp xấu nhất” thuần túy thì quá bi quan vì mọi chính sách đều tệ trên dữ liệu thù địch.

Mô hình bộ nhớ ngoài được hình thức hoá vào cuối thập niên 1980 khi cơ sở dữ liệu trở thành ngành lớn và khoảng cách tốc độ giữa đĩa và bộ nhớ ngày càng rõ. Cache-oblivious ra đời năm 1999[4] với một động cơ rất thực dụng: các thư viện tối ưu theo tham số phần cứng phải viết lại cho mỗi đời máy, và người ta muốn thoát khỏi vòng lặp đó.

Thuật toán luồng dữ liệu có một mốc rõ ràng: bài báo của Alon, Matias và Szegedy giữa thập niên 1990[7] đặt nền cho việc ước lượng các đại lượng thống kê với bộ nhớ nhỏ hơn dữ liệu nhiều bậc. Động cơ ban đầu mang tính lý thuyết, nhưng chỉ một thập kỷ sau, khi lưu lượng mạng và nhật ký hệ thống vượt xa khả năng lưu trữ, các kỹ thuật đó trở thành hạ tầng của mọi hệ thống giám sát.

Điểm chung của bốn câu chuyện: mỗi mô hình mới ra đời khi một giả định của mô hình cũ trở nên sai trong thực tế. Đó là mẫu đáng nhớ nhất của chương, và nó gợi ý rằng sẽ còn những mô hình mới — năng lượng, tính toán trên dữ liệu mã hoá, hay phần cứng chuyên dụng — khi các ràng buộc mới trở nên chi phối.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Giai đoạn đầu của ngành — cận dưới sắp xếp	Có thuật toán sắp xếp nhanh hơn $n \log n$ không?	Lập luận cây quyết định: $\log_2(n!) = \Theta(n \log n)$; và bài học “cận dưới luôn kèm một mô hình”.
1967 — Amdahl	Máy nhiều bộ xử lý có thực sự nhanh hơn tỉ lệ thuận không?	Phân tuần tự không loại bỏ được sẽ chi phối; đặt kỳ vọng thực tế cho tính toán song song.
1985 — Sleator & Tarjan	Đánh giá thế nào cho công bằng một thuật toán không biết tương lai?	Tỉ lệ cạnh tranh: so với lời giải tối ưu biết trước tương lai; ra đời cùng bài báo với splay tree và phân tích khấu hao.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
Cuối thập niên 1980 — mô hình bộ nhớ ngoài	Cơ sở dữ liệu lớn hơn bộ nhớ; đĩa chậm hơn hàng nghìn lần.	Đếm chi phí theo lần chuyển khối; giải thích B-cây và sắp xếp trộn nhiều đường.
Giữa thập niên 1990 — Alon, Matias, Szegedy	Ước lượng thống kê trên dòng dữ liệu quá lớn để lưu.	Nền của thuật toán luồng; một thập kỷ sau trở thành hạ tầng giám sát hệ thống.
1999 — Frigo và cộng sự	Thư viện tối ưu theo tham số phần cứng phải viết lại cho mỗi đời máy.	Cache-oblivious: tối ưu mà không cần biết kích thước khối và bộ nhớ đệm.
2015 — McSherry và cộng sự	Hệ thống phân tán “mở rộng tốt” có thật sự nhanh không?	Nhiều hệ thống bị một chương trình đơn luồng viết tốt đánh bại; nhắc rằng chi phí điều phối không xuất hiện trong phân tích tiệm cận.

1.7 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Lý thuyết thông tin	Entropy và cận dưới cho nén	Cùng lập luận đếm: cần $\log_2 N$ bit để phân biệt N khả năng. Cận dưới sắp xếp và cận dưới nén là hai mặt của một kết quả.
Cơ sở dữ liệu	Sắp xếp ngoài, chỉ mục, tối ưu hoá truy vấn	Mô hình bộ nhớ ngoài là mô hình chính thức của ngành này. Bộ tối ưu truy vấn ước lượng số lần đọc khối chứ không ước lượng số phép toán — đúng tinh thần mục 3.
Hệ điều hành và mạng phân phối nội dung	Chính sách thay thế bộ nhớ đệm	Phân tích cạnh tranh cho biết một chính sách trực tuyến kém tối đa bao nhiêu lần so với biết trước tương lai — thước đo phù hợp hơn “trường hợp xấu nhất” thuần túy.
Hệ thời gian thực và robot	Ràng buộc thời hạn cho từng chu kỳ	Ở đây khâu hao không dùng được và cả tỉ lệ cạnh tranh cũng không đủ: cái cần là cận trên cho <i>mỗi</i> thao tác. Đây là một mô hình chi phí riêng mà sách giáo khoa thuật toán ít nói tới.
Tính toán trên GPU	Mô hình công việc và chiều sâu, hợp nhất truy cập bộ nhớ	Chiều sâu là ràng buộc thật: thuật toán có chuỗi phụ thuộc dài không tăng tốc được dù có hàng nghìn lõi. Ngoài ra kiểu truy cập bộ nhớ quyết định nhiều hơn số phép toán — lại là mô hình bộ nhớ ngoài ở quy mô nhỏ.
Hệ thống dữ liệu lớn	Xử lý luồng, cửa sổ thời gian, sơ đồ xấp xỉ	Ứng dụng trực tiếp của mục 4. Đánh đổi đặc trưng: chấp nhận sai số vài phần trăm để đổi lấy khả năng chạy được ở quy mô mà lời giải chính xác không khả thi.
Thiết bị nhúng, robot chạy pin	Năng lượng làm đơn vị chi phí	Một mô hình gần như vắng mặt trong sách giáo khoa thuật toán nhưng chi phối trong thực tế: thuật toán ít phép toán hơn nhưng truy cập bộ nhớ nhiều hơn có thể tốn nhiều năng lượng hơn.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Vì hai loại mệnh đề có bản chất khác nhau. Chứng minh tồn tại thuật toán nhanh chỉ cần đưa ra *một* đối tượng cụ thể. Chứng minh cận dưới đòi lập luận về *mọi* thuật toán khả dĩ, kể cả những cái chưa ai nghĩ ra — một mệnh đề toàn thể trên một tập vô hạn rất tự do. Vì vậy các cận dưới đã có hầu như đều nằm trong mô hình hạn chế (chỉ so sánh, một lượt đọc), còn trong mô hình

tổng quát thì ta gần như không chứng minh được gì — đó cũng là lý do câu hỏi P đối với NP vẫn mở.

- A2. Không mâu thuẫn, vì hai kết quả nằm trong hai mô hình khác nhau. Cận $\Omega(n \log n)$ chỉ áp cho các thuật toán *chỉ được so sánh phần tử với nhau*; sắp xếp đếm không so sánh mà dùng giá trị làm chỉ số mảng, một thao tác nằm ngoài mô hình đó. Bài học tổng quát: mỗi cận dưới luôn kèm một mô hình, và muốn vượt qua nó thì phải phá vỡ đúng giả định của mô hình — ở đây là “chỉ được so sánh”.
- A3. Vì trong mô hình đó, đơn vị chi phí là *lần chuyển khối*, không phải phép toán. Đọc một khối 4 KB từ đĩa tốn như nhau dù bạn dùng một phần tử hay cả khối, nên thuật toán truy cập tuần tự tận dụng được toàn bộ khối, còn thuật toán nhảy ngẫu nhiên trả giá đầy đủ cho mỗi phần tử. Chênh lệch giữa hai kiểu truy cập có thể tới hàng nghìn lần — đủ để lấn át một khác biệt về số phép toán. Cùng hiện tượng xảy ra ở quy mô nhỏ với cache, và đó là lý do mảng thắng con trỏ (B/06).
- A4. Bằng tỉ lệ cạnh tranh: tỉ số giữa chi phí của thuật toán trực tuyến và chi phí của lời giải tối ưu *biết trước toàn bộ tương lai*. Thước đo này công bằng ở chỗ nó không đòi hỏi thuật toán phải làm điều bất khả thi, mà chỉ hỏi nó tệ hơn bao nhiêu lần so với một đối thủ được biết tương lai. Ví dụ bài thuê hay mua: chiến lược “thuê tới khi tổng tiền thuê bằng giá mua rồi mua” có tỉ lệ 2, và không chiến lược tắt định nào tốt hơn.
- A5. Ba lớp lý do. Một, phần tuần tự không loại bỏ được: nếu 5% chương trình không song song hoá được thì dù vô hạn lõi cũng chỉ nhanh tối đa 20 lần. Hai, chi phí điều phối — tạo luồng, đồng bộ, truyền dữ liệu — tăng theo số lõi và có thể vượt phần được tăng tốc. Ba, tài nguyên chung: nhiều lõi cùng đọc bộ nhớ làm băng thông thành nút cổ chai. Ba lý do này giải thích kết quả thực nghiệm rằng nhiều hệ thống song song thua một chương trình đơn luồng được viết cẩn thận[5].

NÓI LẠI THẬT ĐƠN GIẢN

Chương này nói về hai chuyện, và cả hai đều là chuyện “biết mình đang chơi theo luật nào”. Chuyện thứ nhất: đôi khi ta chứng minh được rằng không ai có thể nhanh hơn một mức nào đó. Ví dụ dễ hình dung nhất: nếu bạn chỉ được phép hỏi “số này lớn hơn số kia không”, thì để sắp xếp n món bạn buộc phải hỏi cỡ $n \log n$ câu, vì mỗi câu hỏi chỉ chia đôi số khả năng còn lại, mà số cách sắp xếp thì rất lớn. Điều quan trọng phải nhớ kèm theo: kết luận đó chỉ đúng *với luật chơi* “chỉ được hỏi so sánh”. Nếu bạn được phép dùng giá trị của món đồ làm địa chỉ ngăn tủ thì luật đã khác và bạn có thể nhanh hơn. Rất nhiều tranh cãi kiểu “sách nói không thể nhanh hơn mà tôi làm được” đến từ việc quên mất luật chơi.

Chuyện thứ hai còn thực dụng hơn: gần như toàn bộ quyển sách này ngầm giả định rằng dữ liệu nằm gọn trong bộ nhớ, đọc lại bao nhiêu lần cũng được, và máy chỉ làm một việc một lúc. Khi một trong ba giả định đó sai, bảng xếp hạng thuật toán đảo lộn.

Nếu dữ liệu nằm trên đĩa, thứ đắt không còn là phép tính mà là mỗi lần với tay xuống kho lấy một thùng — nên thuật toán tốt là thuật toán lấy ít thùng và dùng hết mọi thứ trong thùng. Nếu dữ liệu chảy qua một lần như nước và bạn chỉ có một cái ca nhỏ, bạn phải từ bỏ câu trả lời chính xác và chuyển sang ước lượng. Nếu bạn phải quyết định ngay mà chưa biết tương lai, cách đánh giá công bằng là so với một người biết trước mọi chuyện — và bạn chấp nhận tệ hơn họ vài lần. Còn nếu bạn có nhiều lõi thì hãy nhớ: chuỗi việc phải làm nối tiếp nhau sẽ quyết định thời gian, chứ không phải tổng số việc.

Một câu: Mọi kết luận về “thuật toán nào tốt hơn” đều là kết luận trong một luật chơi; đổi luật thì đổi câu trả lời.

Chỗ không rút gọn được: Chứng minh cận dưới trong mô hình tính toán tổng quát vẫn nằm ngoài khả năng hiện nay; đó là lý do nhiều câu hỏi lớn còn mở.

Thuật ngữ / câu hỏi	Trả lời
●●● Cận dưới luôn kèm mô hình	$\Omega(n \log n)$ cho sắp xếp chỉ đúng trong mô hình <i>so sánh</i> ; sắp xếp đếm không mâu thuẫn vì nó dùng giá trị làm chỉ số.
●●● Ba kỹ thuật cận dưới	Cây quyết định (số lá $\geq$ số kết quả); lập luận đối thủ (giữ nhiều khả năng càng lâu càng tốt); đếm thông tin ($\log_2 N$ bit).
●●● Vì sao cận dưới khó	Phải lập luận về <i>mọi</i> thuật toán khả dĩ, kể cả chưa ai nghĩ ra — mệnh đề toàn thể trên tập vô hạn tự do.
●●● Mô hình bộ nhớ ngoài	Đếm số lần chuyển khối, không đếm phép toán; sắp xếp là $O(\frac{n}{B} \log_{M/B} \frac{n}{B})$ lần chuyển. Giải thích B-cây và trộn nhiều đường.
●●● Hai đại lượng của song song	Tổng công việc và chiều sâu (chuỗi phụ thuộc dài nhất). Thời gian bị chặn dưới bởi chiều sâu, dù có bao nhiêu lõi.
●●● Ba lý do song song không tăng tuyến tính	Phản tuần tự chi phối; chi phí điều phối tăng theo số lõi; tranh chấp băng thông bộ nhớ.
●● Tỷ lệ cạnh tranh	So chi phí của thuật toán trực tuyến với lời giải tối ưu biết trước tương lai; bài thuê hay mua có tỉ lệ 2 và đó là tối ưu cho chiến lược tất định.
●● Mô hình luồng dữ liệu	Một lượt đọc, bộ nhớ nhỏ hơn dữ liệu nhiều bậc; hầu hết bài chính xác bất khả thi nên chuyển sang xấp xỉ (HyperLogLog, sơ đồ đếm, lấy mẫu hồ chứa).
●● Cache-oblivious	Tối ưu số lần chuyển khối mà không cần biết B và M ; đạt được bằng chia để trị đệ quy tới mọi tỉ lệ.
●● Độ phức tạp tinh	Cận dưới <i>có điều kiện</i> : “nếu bài A không nhanh hơn được thì B cũng không”; cách sống chung với việc cận dưới tuyệt đối quá khó.
●● Khi lý thuyết mâu thuẫn thực nghiệm	Nghi mô hình chi phí trước, đừng nghi phép đo — bạn có thể đang ở mô hình khác với mô hình mà kết quả lý thuyết giả định.
● Hệ thời gian thực	Cần cận trên cho <i>mỗi</i> thao tác; khấu hao và tỉ lệ cạnh tranh đều không đủ.
●● Cận dưới có điều kiện	Neo vào SETH hoặc 3SUM rồi quy dẫn: khoảng cách chính sửa không thể tính trong $O(n^{2-\epsilon})$ trừ khi SETH sai. Giá trị thực hành là biết khi nào nên ngừng tìm thuật toán nhanh hơn.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Cận dưới có ích gì cho công việc hằng ngày của tôi?

| Chủ yếu ở hai chỗ. Thứ nhất, nó cho bạn biết khi nào nên ngừng tìm — nếu bài của bạn quy về sắp xếp thì đừng tìm lời giải tuyến tính trong mô hình so sánh. Thứ hai, và hữu ích hơn, nó chỉ ra *giả định nào cần phá* để đi nhanh hơn: khoá có phải số nguyên nhỏ không (thì dùng sắp xếp đếm), dữ liệu có gần như đã sắp không (thì có thuật toán thích ứng), bạn có chấp nhận kết quả xấp xỉ không. Cận dưới không chỉ nói “không được” mà nói “không được *trong luật này*”.

2. Làm sao biết chương trình của tôi đang bị chặn bởi bộ nhớ chứ không phải bởi tính toán?

| Hai kiểm tra rẻ. Một: giảm kích thước dữ liệu xuống mức vừa trong bộ nhớ đệm và xem thời gian trên mỗi phần tử có giảm mạnh không — nếu có, bạn đang bị chặn bởi bộ nhớ. Hai: thay đổi *thứ tự truy cập* mà giữ nguyên số phép toán, ví dụ duyệt ma trận theo hàng thay vì theo cột; nếu thời gian đổi nhiều lần thì chi phí thật nằm ở truy cập. Đây là hai thí nghiệm nên làm trước khi tối ưu bất cứ thứ gì (E/26).

3. Thuật toán trực tuyến có xuất hiện trong bài tập không, hay chỉ là lý thuyết?

| Có, dưới hai dạng. Dạng thứ nhất là các bài yêu cầu trả lời truy vấn ngay mà không được xử lý ngoại tuyến — lúc đó các kỹ thuật như thuật toán Mo (B/10) không dùng được và bạn phải chọn cấu trúc dữ liệu khác. Dạng thứ hai là các bài mô phỏng chính sách quyết định, ví dụ quản lý bộ đệm.

Ngoài thi đấu thì đây là mô hình rất thực: mọi hệ thống phải quyết định khi chưa biết yêu cầu tiếp theo.

4. Bộ nhớ đệm LRU có tối ưu không?

| Không tối ưu, và người ta biết chính xác nó kém bao nhiêu. Chính sách tối ưu là loại bỏ trang sẽ được dùng lại xa nhất trong tương lai — không cài được vì cần biết tương lai. LRU có tỉ lệ cạnh tranh bằng kích thước bộ đệm trong trường hợp xấu nhất, nhưng trong thực tế nó chạy tốt vì các mẫu truy cập thật có tính cục bộ thời gian. Đây là ví dụ điển hình cho việc phân tích trường hợp xấu nhất quá bi quan, và là lý do các phân tích tinh hơn ra đời.

5. Nếu tôi muốn tăng tốc bằng song song, nên bắt đầu từ đâu?

| Bắt đầu bằng việc tính *chiều sâu* của thuật toán: nếu nó có một chuỗi phụ thuộc dài thì song song hoá không cứu được và bạn cần đổi thuật toán trước. Sau đó chọn nơi song song hoá ở mức thô nhất có thể — các nhánh trên cùng của chia để trị, các phần tử độc lập của một vòng lặp lớn — vì song song ở mức mịn thường bị chi phí điều phối nuốt hết. Cuối cùng, luôn đo trước và sau; kết quả thường khác dự đoán, đúng như bài học COST.

6. Thuật toán xấp xỉ trong mô hình luồng đáng tin tới đâu?

| Chúng đi kèm bảo đảm dạng “sai số tương đối không quá ε với xác suất ít nhất $1 - \delta$ ”, và bạn chọn ε, δ bằng cách trả thêm bộ nhớ. Đây là bảo đảm chứng minh được, không phải quan sát thực nghiệm — nên chúng đáng tin theo đúng nghĩa đã phát biểu. Điều cần cẩn thận trong thực tế: các bảo đảm này thường giả định các hàm băm đủ ngẫu nhiên và các phần tử của luồng không phụ thuộc vào lựa chọn ngẫu nhiên của bạn ($C/20$).

7. Có mô hình chi phí nào đang trở nên quan trọng mà sách giáo khoa chưa nói nhiều?

| Ít nhất hai. Năng lượng: trên thiết bị chạy pin và trong trung tâm dữ liệu, joule mỗi thao tác là ràng buộc thật, và nó không tỉ lệ với số phép toán — truy cập bộ nhớ tốn năng lượng hơn phép tính rất nhiều. Và tính toán trên dữ liệu được bảo vệ: khi phải tính trên dữ liệu mã hoá hoặc phân tán mà không lộ thông tin, chi phí của một phép toán thay đổi hoàn toàn, và các thuật toán tối ưu cho mô hình đó rất khác. Cả hai đều là ví dụ cho mẫu ở mục 6: mô hình mới ra đời khi một ràng buộc mới trở nên chi phối.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Chứng minh cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh	[8]	Cây quyết định cộng công thức Stirling; ba câu là đủ.
● Chứng minh cần ít nhất $n - 1$ phép so sánh để tìm cực đại	Bài kinh điển	Lập luận đối thủ: mỗi phép so sánh loại tối đa một ứng viên.
●● Tìm đồng thời cực đại và cực tiểu với $\lceil 3n/2 \rceil - 2$ phép so sánh	Bài kinh điển	So từng cặp trước rồi mới so với kỷ lục; chứng minh cận dưới bằng lập luận đối thủ.
●● Sắp xếp một tệp lớn hơn bộ nhớ	Bài tập hệ thống	Trộn nhiều đường với heap (B/07); đo số lần đọc/ghi và so với dự đoán của mô hình bộ nhớ ngoài.
●● Đếm số phần tử phân biệt trong luồng bằng bộ nhớ nhỏ	[9]	Cài HyperLogLog đơn giản; đo sai số thực tế trên dữ liệu vài triệu phần tử.
●● Mô phỏng LRU và chính sách tối ưu trên cùng dãy truy cập	Bài tập thực nghiệm	Đo tỉ số trượt bộ đệm; đối chiếu với khái niệm tỉ lệ cạnh tranh.
●●● Nhân ma trận theo khối và đo hiệu ứng cache	Bài tập thực nghiệm	So ba cách: ngây thơ, đối thứ tự vòng lặp, chia khối. Số liệu sẽ nói rõ hơn mọi lý thuyết.
●●● Cài phiên bản cache-oblivious của một thuật toán quen thuộc	[4]	Ví dụ: chuyển vị ma trận bằng chia để trị; so với bản ngây thơ trên ma trận lớn.
●●● Phân tích bài thuê hay mua và chứng minh tỉ lệ 2 là tối ưu	Bài kinh điển	Dùng lập luận đối thủ: đối thủ chọn thời điểm dừng ngay sau khi bạn quyết định mua.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Đo tăng tốc thực tế của một chương trình song song theo số lõi	Bài tập thực nghiệm	Vẽ đường tăng tốc và ước lượng tỉ lệ phần tuần tự; đối chiếu với ba lý do ở mục 5.
●●● So sánh một hệ thống xử lý dữ liệu song song với một chương trình đơn luồng viết cẩn thận	[5]	Chọn một bài đồ thị vừa phải; kết quả thường gây ngạc nhiên và là bài học đáng nhớ nhất chương.

Kết chương và đọc tiếp

Phần D khép lại với một thông điệp thống nhất: mọi kết luận về hiệu năng đều là kết luận *trong một mô hình*. Chương D/23 nói về giới hạn do bản chất bài toán; chương này nói về giới hạn chứng minh được trong mô hình hạn chế, và về những mô hình khác nơi bảng xếp hạng thuật toán đảo lộn. Ba điều đáng mang theo: cận dưới luôn kèm mô hình và muốn vượt qua thì phải phá đúng giả định; đơn vị chi phí quyết định thuật toán tối ưu; và khi lý thuyết mâu thuẫn thực nghiệm, hãy nghi mô hình trước.

Phần E chuyển hẳn sang thực hành. Bốn chương cuối nói về khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng, đúng hạn”: cài đặt và gỡ lỗi (E/25), thuật toán trong hệ thống thật (E/26), chiến thuật phỏng vấn và thi đấu (E/27), và cuối cùng là lộ trình luyện tập kèm ngân hàng bài tập để mang theo sau khi gấp sách (E/28).

I Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Backurs & Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False)”	SIAM Journal on Computing, 2018 (bản hội nghị STOC 2015)	Đọc phần mở đầu và sơ đồ quy dẫn từ bài vectơ trực giao; đây là ví dụ mẫu mực của cận dưới có điều kiện.
nền tảng	Impagliazzo & Paturi, “On the Complexity of k -SAT”	Journal of Computer and System Sciences, 2001	Nơi ETH và SETH được phát biểu. Đọc để biết chính xác giả thuyết neo nói gì — và không nói gì.
tra khi cần	Gajentaan & Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry”	Computational Geometry: Theory and Applications, 1995	Danh mục các bài 3SUM-khó trong hình học; tra khi bài của bạn có mùi “ba đối tượng thẳng hàng”.
tra khi cần	Frigo, Leiserson, Prokop, Ramachandran, “Cache-Oblivious Algorithms”	FOCS, 1999	Mô hình chi phí thay thế cho mô hình RAM, dùng cho mục 3 của chương này.
tra khi cần	Borodin & El-Yaniv, <i>Online Computation and Competitive Analysis</i>	Cambridge University Press, 1998	Sách chuẩn cho tỉ lệ cạnh tranh ở mục 4; đọc chương về bài toán thuê hay mua và về bộ nhớ đệm.

Nguồn

- [1] R. Impagliazzo and R. Paturi, “On the Complexity of k-SAT,” *Journal of Computer and System Sciences*, vol. 62, no. 2, pp. 367–375, 2001.
- [2] A. Backurs and P. Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False),” *SIAM Journal on Computing*, vol. 47, no. 3, 2018, bản hội nghị: STOC 2015.
- [3] A. Gajentaan and M. H. Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry,” *Computational Geometry: Theory and Applications*, vol. 5, no. 3, pp. 165–185, 1995.
- [4] M. Frigo, C. E. Leiserson, H. Prokop, et al., “Cache-Oblivious Algorithms,” in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 1999, pp. 285–297.
- [5] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [6] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [7] N. Alon, Y. Matias, and M. Szegedy, “The Space Complexity of Approximating the Frequency Moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [9] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.